

PROJECT DOCUMENTATION



ShadowDeploy

AI-Powered Kubernetes Incident Response Platform

VERSION 1.0 · WINDOWS 11 · 100% LOCAL — ZERO CLOUD COST

Python 3.11 | Kubernetes | Docker | Ollama | FastAPI | Prometheus | Grafana | Helm | Jenkins

PREPARED BY

Sanskruti Chavan

Junior Cloud Engineer | AIOps Enthusiast

Pranav Chaudhari

DevOps Engineer | AIOps Enthusiast

Built entirely on free & open-source tools · No cloud APIs · No subscription cost

Table of Contents

Section No.	Title	Page No.
1	Introduction	3
2	Project Objective	4
3	Tools & Technologies Used	5
4	High-Level Architecture	7
5	Project Workflow — Step by Step	8
5.1	Phase 1 — Base Infrastructure	8
5.2	Phase 2 — Intentionally Failing Test Applications	9
5.3	Phase 3 — Kubernetes Watcher Agent	9
5.4	Phase 4 — Log Collector	9
5.5	Phase 5 — Ollama AI Analyzer	10
5.6	Phase 6 — FastAPI Backend + SQLite Database	10
5.7	Phase 7 — Email Alert System	11
5.8	Phase 8 — Prometheus + Grafana Dashboard	12
6	GitHub Setup & Repository Structure	14
7	End-to-End Automation Summary	16
8	Results and Discussion	18
9	Conclusion	22

1. Introduction

ShadowDeploy is a self-hosted, privacy-first, AI-powered Kubernetes incident response platform built entirely with free and open-source tools. It was designed to solve one of the most common and time-consuming problems in modern DevOps: identifying why a Kubernetes pod failed and what to do about it.

In production environments, pod failures can cascade quickly. A single misconfigured environment variable can bring down a payment service. A memory leak can OOMKill a critical microservice at 3am. Engineers currently spend hours manually collecting logs, searching error messages, and piecing together root causes — often under pressure.

ShadowDeploy automates this entire process:

- It watches the Kubernetes cluster continuously for pod failures
- It collects pod logs automatically the moment a failure is detected
- It sends those logs to a locally-running Large Language Model (LLM) via Ollama
- The LLM analyzes the logs and returns a structured root cause analysis
- The analysis is saved to a database and an email alert is sent immediately
- Everything is visualized on a live Grafana dashboard

Key differentiator: Unlike cloud-based AIOps tools, ShadowDeploy runs 100% on your local machine. No OpenAI API key required. No data leaves your laptop. No monthly subscription. Zero cloud cost.

This document covers the complete technical architecture, all implementation phases, the GitHub repository structure, and the end-to-end automation flow of the ShadowDeploy platform.

2. Project Objective

The primary objective of ShadowDeploy is to build a production-grade AIOps incident response system that demonstrates the convergence of Kubernetes, local LLMs, observability, and automation — all running on a developer's local machine.

2.1 Problem Statement

Every DevOps team faces the same challenge when pods fail in Kubernetes:

- Pod logs are scattered across multiple containers and restarts
- Engineers must manually run `kubectl describe`, `kubectl logs --previous`, and `kubectl get events`
- Root cause analysis (RCA) requires deep knowledge of the specific error pattern
- Time-to-resolution (MTTR) is measured in hours, not minutes
- Existing commercial AIOps tools are expensive and send sensitive log data to the cloud

2.2 Solution Goals

ShadowDeploy was built to achieve the following specific goals:

- Automatically detect Kubernetes pod failures within 10 seconds of occurrence
- Collect relevant logs without any manual `kubectl` commands
- Generate human-readable root cause analysis using a local LLM — no cloud APIs
- Provide a specific, actionable fix and rollback command for every incident
- Store all incidents persistently in a queryable REST API
- Alert engineers immediately via HTML email with colour-coded severity
- Provide a live Grafana dashboard showing incident trends and severity breakdown
- Run completely locally with zero external dependencies or API costs

2.3 Target Audience

ShadowDeploy is built for:

- Site Reliability Engineers (SREs) managing Kubernetes clusters
- DevOps Engineers building internal tooling for incident management
- Platform Engineers exploring AIOps and LLMOps integration
- DevOps learners wanting a portfolio project that demonstrates real-world skills

3. Tools & Technologies Used

ShadowDeploy uses eight categories of technology, all free and open-source. Every tool was chosen specifically to reflect real enterprise DevOps stacks.

Category	Tool / Technology	Purpose
Container Orchestration	Kubernetes (Minikube v1.37)	Local K8s cluster for pod management
Containerization	Docker Desktop	Build and run all application containers
CI/CD	Jenkins (Docker image)	Pipeline automation and deployment triggers
Language & Runtime	Python 3.11	All agent, API, and alert code
API Framework	FastAPI + Uvicorn	REST API for incident storage and retrieval
Local AI Engine	Ollama	Serve LLMs locally without cloud APIs
AI Model	Mistral 7B / Llama 3	Local language model for root cause analysis
Database	SQLite	Lightweight embedded incident storage
Data Validation	Pydantic v2	Request/response schema validation in FastAPI
Monitoring	Prometheus	Metrics collection via /metrics endpoint
Visualization	Grafana	Live incident dashboard with PromQL queries
Package Manager	Helm v3	Install Prometheus + Grafana into Minikube
Alerting	smtplib (Python)	HTML email alerts via Gmail SMTP
K8s SDK	kubernetes-python	Programmatic access to Kubernetes API
Env Config	python-dotenv	Load .env secrets into config.py
OS / Platform	Windows 11 + WSL2	Development environment
Version Control	Git + GitHub	Source control and portfolio hosting

3.1 Why Ollama?

Ollama was chosen as the local LLM runtime for four specific reasons:

- **Privacy:** Pod logs often contain database connection strings, API keys, and internal hostnames. Sending these to OpenAI or any third-party API is a security risk. Ollama keeps everything on-device.
- **Cost:** Zero API cost. No per-token pricing. No rate limits. Run unlimited analyses.
- **Offline capability:** Works in air-gapped environments, VPNs, and restricted networks — common in enterprise settings.
- **Trend alignment:** Ollama is one of the fastest-growing tools in the local AI space, making it highly recognizable to technical hiring managers.

3.2 Why FastAPI?

FastAPI was chosen over Flask or Django for three reasons: automatic OpenAPI/Swagger documentation at /docs, native async support for concurrent requests, and Pydantic-based schema validation that prevents malformed incident data from being stored.

4. High-Level Architecture

ShadowDeploy consists of four main layers that work together in a continuous event-driven pipeline:

LAYER 1 — Detection Layer: The Kubernetes Watcher Agent continuously polls the K8s API every 10 seconds, monitoring all pod statuses in the default namespace.

LAYER 2 — Analysis Layer: When a failure is detected, the Log Collector grabs pod logs and the AI Analyzer sends them to Ollama for structured root cause analysis using a carefully engineered prompt.

LAYER 3 — Storage & Alert Layer: The FastAPI backend saves every incident to SQLite. The Email Sender dispatches an HTML alert with severity colour-coding, fix steps, and rollback command.

LAYER 4 — Observability Layer: Prometheus scrapes the /metrics endpoint every 15 seconds. Grafana visualizes incident counts, severity breakdown, and failure trends in real time.

4.1 Complete Data Flow

- Step 1: Jenkins pipeline deploys a new version of an application
- Step 2: Kubernetes schedules the pod and attempts to start the container
- Step 3: Pod enters a failure state (CrashLoopBackOff / OOMKilled / ImagePullBackOff etc.)
- Step 4: ShadowDeploy Watcher detects the failure within 10 seconds
- Step 5: Log Collector runs `kubectl logs --previous` to capture crash output
- Step 6: AI Analyzer builds a structured prompt and sends it to Ollama (localhost:11434)
- Step 7: Mistral/Llama3 returns: Root Cause, Severity, Fix, Rollback Command, Prevention
- Step 8: FastAPI receives a POST /api/incidents call and saves to SQLite
- Step 9: Email Sender dispatches an HTML email with the full AI analysis
- Step 10: Prometheus scrapes /metrics, Grafana dashboard updates within 30 seconds

4.2 Failure Types Detected

Failure Type	Description	Default Severity
CrashLoopBackOff	App crashes on startup, K8s keeps restarting	CRITICAL
OOMKilled	Container exceeds memory limit, kernel kills it	HIGH
ImagePullBackOff	Docker image not found or inaccessible	CRITICAL
Missing Env Variable	Required config (DB_HOST, API_KEY) not set	CRITICAL
Failed Health Check	Liveness probe hits wrong port, restarts pod	MEDIUM

5. Project Workflow — Step by Step

ShadowDeploy was built across 8 sequential phases, each adding a layer of functionality to the system. The table below summarises all phases:

#	Phase	What Was Built	Status
1	Base Infrastructure	Docker, Minikube, Python venv, Ollama model pull	Complete
2	Failing Test Apps	5 Docker apps that crash in different ways + K8s manifests	Complete
3	Kubernetes Watcher	Python agent polling K8s API every 10s for pod failures	Complete
4	Log Collector	Automatic pod log capture on failure detection	Complete
5	Ollama AI Analyzer	Local LLM integration for root cause analysis	Complete
6	FastAPI + SQLite	REST API with 8 endpoints, SQLite incident storage	Complete
7	Email Alerts	HTML email with colour-coded severity via smtplib	Complete
8	Prometheus + Grafana	Metrics endpoint + Helm install + dashboard import	Complete

5.1 Phase 1 — Base Infrastructure

The first phase establishes the complete local development environment on Windows 11.

What was installed:

- Docker Desktop with WSL2 backend — container runtime for Minikube and application images
- Minikube v1.37 — single-node Kubernetes cluster running with 4 CPUs and 6GB RAM via the Docker driver
- kubectl — command-line tool for interacting with the Kubernetes cluster
- Python 3.11 with virtual environment (venv) — isolated dependency management
- Ollama — local LLM runtime, with Mistral 7B model pulled (4.1GB)
- Helm v3 — Kubernetes package manager for Prometheus and Grafana installation

Key commands:

```
minikube start --cpus=4 --memory=6144 --driver=docker

python -m venv venv && venv\Scripts\activate

pip install fastapi uvicorn kubernetes requests python-dotenv prometheus-client aiofiles

ollama pull mistral
```

5.2 Phase 2 — Intentionally Failing Test Applications

Five small Docker applications were built to simulate real-world Kubernetes failures. These give the watcher real failures to detect and the AI real logs to analyze.

- App 1 — missing-env-app: Reads DB_HOST and API_KEY on startup. With no env vars set in the deployment, it exits immediately with a FATAL error. This triggers CrashLoopBackOff.
- App 2 — crashloop-app: Starts successfully then crashes after 3-8 seconds with a random simulated error (ConnectionRefused, MemoryError, OSError). Continuously restarts.
- App 3 — oom-kill-app: Allocates 1MB of memory every 0.5 seconds in an infinite loop. With a 64MB memory limit set in the manifest, Kubernetes OOMKills it after ~60 seconds.
- App 4 — bad-image-app: References a Docker image that doesn't exist (this-image-does-not-exist-anywhere:v999). Kubernetes immediately shows ImagePullBackOff.
- App 5 — bad-port-app: HTTP server runs on port 5000 but the Kubernetes liveness probe is configured to check port 8080. Probe fails repeatedly, pod is restarted.

Critical Windows step: Images must be built inside Minikube's Docker daemon using: `minikube docker-env --shell powershell | Invoke-Expression` before running `docker build` commands. Without this, all pods will show ImagePullBackOff even for locally built images.

5.3 Phase 3 — Kubernetes Watcher Agent

The watcher is the core of ShadowDeploy. It is a Python process running an infinite loop that polls the Kubernetes API every 10 seconds and inspects every pod in the cluster.

Detection logic:

For each pod, the watcher inspects `container_statuses` from the Kubernetes API response. It checks both the waiting state (CrashLoopBackOff, ImagePullBackOff) and the terminated state (OOMKilled, Error) against a set of 8 known failure reasons.

A `seen_failures` dictionary tracks which pods have already triggered an alert, preventing duplicate notifications on subsequent poll cycles. When a pod recovers, it is removed from the dictionary and a recovery log is printed.

Key file: *agent/watcher.py*

```
FAILURE_REASONS = { 'CrashLoopBackOff', 'OOMKilled', 'Error', 'ImagePullBackOff',
                    'ErrImagePull', 'CreateContainerConfigError', 'RunContainerError' }

# Run from project root:
python -m agent.watcher
```

5.4 Phase 4 — Log Collector

The log collector is called immediately after a failure is detected. It retrieves the last 80 lines of logs from the failing pod using the Kubernetes Python SDK.

Collection strategy — two-pass approach:

- Pass 1: Attempts to collect logs from the previous container instance using `previous=True`. This captures the actual crash output before Kubernetes restarted the container.
- Pass 2: Falls back to current container logs if no previous logs exist (e.g. first crash or `ImagePullBackOff` where no container ever ran).

The 80-line limit balances providing enough context for the AI to analyze while keeping the prompt size manageable (under 3000 characters sent to Ollama).

5.5 Phase 5 — Ollama AI Analyzer

The AI analyzer is the most technically distinctive component of ShadowDeploy. It constructs a structured prompt from the failure data and sends it to Ollama's local REST API at `localhost:11434`.

Prompt engineering:

The prompt is designed to produce deterministic, parseable output by instructing the model to respond in exactly five labelled sections: ROOT CAUSE, SEVERITY, PROBABLE FIX, ROLLBACK COMMAND, and PREVENTION. Temperature is set to 0.1 (near-deterministic) to ensure consistent structured output rather than creative variation.

API call configuration:

```
POST http://localhost:11434/api/generate
{
  "model": "mistral",
  "prompt": "<structured prompt>",
  "stream": false,
  "options": { "temperature": 0.1, "num_predict": 400 }
}
```

Response parsing:

The `_parse_response()` function uses line-by-line parsing to extract each section by its label prefix. This is more robust than regex for LLM output, which may have slight formatting variations between model versions.

Fallback behaviour:

If Ollama is unreachable or times out, the analyzer returns a safe fallback dict with the K8s failure reason and a generic `kubectl rollback` command, ensuring the watcher continues running even without AI analysis.

5.6 Phase 6 — FastAPI Backend + SQLite Database

A full REST API was built using FastAPI to store, retrieve, and query incidents. SQLite was chosen for zero-configuration embedded storage that requires no separate database server.

Database schema:

The incidents table stores 14 fields including `pod_name`, `namespace`, `failure_reason`, `root_cause`, `severity`, `fix`, `rollback_cmd`, `prevention`, `raw_logs`, `raw_ai`, `restarts`, `resolved` flag, `created_at` and `resolved_at` timestamps.

API endpoints:

Method	Endpoint	Description	Response
POST	/api/incidents	Watcher saves a new incident	201 Created
GET	/api/incidents	List all incidents (newest first)	200 + JSON array
GET	/api/incidents/{id}	Get one incident with full AI output	200 + JSON object
PATCH	/api/incidents/{id}/resolve	Mark an incident as resolved	200 OK
GET	/api/stats	Aggregate counts by severity/reason	200 + stats JSON
GET	/health	Liveness check + DB connectivity	200 OK
GET	/metrics	Prometheus scrape endpoint	Plain text metrics
GET	/docs	Auto-generated Swagger UI	Browser HTML

The FastAPI server also exposes a /metrics endpoint using the prometheus-client library, which reads the latest incident counts from SQLite and converts them into Prometheus gauge metrics on every scrape.

Run the API:

```
uvicorn api.main:app --reload --port 8000

# Interactive docs: http://localhost:8000/docs
```

5.7 Phase 7 — Email Alert System

The email alert system sends a fully formatted HTML email to a configured recipient every time a pod failure is detected and analyzed. It uses Python's built-in smtplib library — no third-party email library required.

Email design:

- Header: Colour-coded by severity — red for CRITICAL, orange for HIGH, amber for MEDIUM, green for LOW
- Metadata table: Pod name, K8s failure reason, restart count, detection timestamp
- Root Cause section: Red left-border highlight with AI-generated explanation
- Probable Fix section: Green left-border highlight with numbered action steps
- Rollback Command section: Dark code block with the exact kubectl command
- Prevention section: Blue left-border highlight with architectural recommendation

Gmail configuration:

Gmail requires an App Password (16-character code from myaccount.google.com/apppasswords) rather than the regular account password. This is because Google blocks direct SMTP login with account passwords as a security policy.

```
EMAIL_SENDER=your.gmail@gmail.com
EMAIL_APP_PASSWORD=abcdefghijklmnop # 16-char app password, no spaces
EMAIL_RECEIVER=any.inbox@gmail.com # can be same or different address
```

Startup confirmation:

A startup email is sent when the watcher initializes, confirming the email configuration is correct before any failures occur. This prevents discovering a misconfigured email at 3am when the first real incident happens.

5.8 Phase 8 — Prometheus + Grafana Dashboard

The final phase adds professional observability to ShadowDeploy using Prometheus for metrics collection and Grafana for visualization.

Prometheus setup:

Prometheus is installed inside the Minikube cluster using Helm. A custom values file configures it to scrape the FastAPI /metrics endpoint at host.minikube.internal:8000 — the special hostname Minikube uses on Windows to reach the host machine's localhost.

```
helm install prometheus prometheus-community/prometheus \
  --namespace monitoring \
  --values monitoring/prometheus-values.yaml
```

Prometheus metrics exposed:

- shadowdeploy_incidents_total — total incident count (gauge)
- shadowdeploy_incidents_open — unresolved incidents (gauge)
- shadowdeploy_incidents_by_severity{severity=CRITICAL/HIGH/MEDIUM/LOW} — per-severity counts
- shadowdeploy_incidents_by_reason{reason=CrashLoopBackOff/...} — per-failure-type counts
- shadowdeploy_pod_restarts_total — cumulative restarts across all incidents

Grafana dashboard panels:

- Stat panel: Total Incidents (red background, updates in real time)
- Stat panel: Open Incidents (orange, shows unresolved count)
- Stat panel: Total Pod Restarts (blue, cumulative restart counter)
- Stat panel: Critical Incidents (dark red, CRITICAL severity count)
- Bar gauge: Incidents by Failure Reason (horizontal, colour-coded per reason)
- Pie chart: Incidents by Severity (percentage breakdown)
- Time series: Incident Rate Over Time (total, open, and critical trend lines)

```
helm install grafana grafana/grafana --namespace monitoring \  
  --set service.type=NodePort --set adminPassword=shadowdeploy123  
  
minikube service grafana --namespace monitoring    # opens in browser
```

6. GitHub Setup & Repository Structure

6.1 Repository Structure

```
shadowdeploy/
├── agent/
│   ├── __init__.py          # Python package marker
│   ├── watcher.py           # Kubernetes pod failure watcher (core loop)
│   ├── log_collector.py     # Collects pod logs on failure
│   ├── ai_analyzer.py       # Ollama LLM integration for RCA
│   ├── alert_sender.py      # HTML email alerts via smtplib
│   └── metrics.py           # Prometheus gauge metrics
├── api/
│   ├── __init__.py
│   ├── main.py              # FastAPI app entry point
│   ├── database.py          # SQLite connection and operations
│   ├── models.py            # Pydantic request/response models
│   └── routes/
│       ├── incidents.py     # GET + POST /api/incidents
│       ├── stats.py         # GET /api/stats
│       └── health.py        # GET /health
├── apps/
│   ├── missing-env/         # App 1: crashes on missing env var
│   ├── crashloop/          # App 2: random crashes
│   ├── oom-kill/            # App 3: memory leak
│   └── bad-port/            # App 5: wrong health check port
├── k8s/
│   ├── failing-apps/        # Kubernetes manifests for all 5 failures
│   └── shadowdeploy/        # Manifests for the ShadowDeploy agent itself
├── monitoring/
│   ├── prometheus-values.yaml
│   └── grafana-dashboard.json
├── database/                # SQLite .db file (git-ignored)
├── config.py                # Central config – reads .env
├── requirements.txt          # All Python dependencies
├── .env                     # Secrets – NEVER committed to git
├── .env.example             # Safe template for contributors
├── .gitignore
└── README.md
```

6.2 Git Setup Commands

```
git init && git branch -M main
git add .
git status    # verify .env is NOT listed
git commit -m "feat: ShadowDeploy v1.0 - AI-powered K8s incident response"
git remote add origin https://github.com/YOUR_USERNAME/shadowdeploy.git
git push -u origin main
```

6.3 Critical .gitignore Entries

SECURITY WARNING: Never commit .env to GitHub. Your Gmail App Password in .env gives full access to your Gmail account. Verify it is in .gitignore and not staged before every git push.

```
.env
venv/
__pycache__/
*.pyc
database/*.db
*.log
```

6.4 Recommended GitHub Repository Settings

- Repository name: shadowdeploy (lowercase, no spaces)
- Description: AI-powered Kubernetes incident response platform using local LLMs — zero cloud cost
- Visibility: Public (essential for portfolio / recruiter visibility)
- Topics: kubernetes, devops, aiops, llm, ollama, fastapi, prometheus, grafana, python, incident-management, sre, platformengineering
- Pin this repository to your GitHub profile page
- Add a screenshot of the Grafana dashboard as docs/dashboard.png and reference it in the README

7. End-to-End Automation Summary

The following describes the complete automated flow from a deployment trigger to a Grafana dashboard update — with no manual intervention required at any step.

T+0s Step 1: Deployment Trigger

A Jenkins pipeline (or manual `kubectl` apply) deploys a new version of an application to the Minikube cluster. In this case, the `missing-env-app` deployment is applied without the required `DB_HOST` environment variable.

T+2s Step 2: Pod Failure

Kubernetes schedules the pod, Docker pulls the image (already cached in Minikube), and the container starts. The Python application immediately checks for `DB_HOST`, finds it missing, prints 'FATAL ERROR: `DB_HOST` not set', and calls `sys.exit(1)`. Kubernetes sees the non-zero exit code and marks the pod as Failed.

T+10s Step 3: Watcher Detection

On its next polling cycle (every 10 seconds), the ShadowDeploy watcher calls `list_namespaced_pod()` via the Kubernetes Python SDK. It finds `missing-env-app` with `container_status.state.waiting.reason == 'CrashLoopBackOff'`. Since this pod is not in `seen_failures`, `handle_failure()` is called.

T+10s Step 4: Log Collection

`log_collector.py` calls `read_namespaced_pod_log()` with `previous=True` and `tail_lines=80`. It retrieves the crash output from the container's previous run: the FATAL ERROR message and stack trace. These 187 characters are returned as a string.

T+11s Step 5: AI Analysis Request

`ai_analyzer.py` constructs a structured prompt embedding the pod name, failure reason, and log output. It sends a POST request to `http://localhost:11434/api/generate` with `model=mistral`, `stream=false`, and `temperature=0.1`.

T+30s Step 6: AI Analysis Response

Ollama responds after approximately 20 seconds (first call loads the model; subsequent calls are 5-10 seconds). The response contains five structured sections. `_parse_response()` extracts: ROOT CAUSE (`DB_HOST` not set), SEVERITY (CRITICAL), FIX (add env var to `deployment.yaml`), ROLLBACK (`kubectl rollout undo`), PREVENTION (use K8s Secrets).

T+30s Step 7: Incident Saved to API

`save_to_api()` sends a POST request to `http://localhost:8000/api/incidents` with the complete incident payload. FastAPI validates it via the IncidentCreate Pydantic model and calls `save_incident()` which inserts a row into the SQLite incidents table. The API returns `{"id": 6, "status": "saved"}`.

T+31s Step 8: Email Alert Dispatched

`send_alert()` constructs an HTML email with a red CRITICAL header, a metadata table, and four colour-coded sections for the AI analysis. It connects to `smtp.gmail.com:587`, authenticates with the App Password via STARTTLS, and sends the email to the configured recipient.

T+45s Step 9: Prometheus Scrape

On its next 15-second scrape cycle, Prometheus hits `http://host.minikube.internal:8000/metrics`. The endpoint calls `refresh_metrics()` which reads the latest counts from SQLite and updates the Prometheus gauges. Prometheus stores the new time series data point.

T+75s Step 10: Grafana Dashboard Updates

Grafana's 30-second auto-refresh queries Prometheus using PromQL. The Total Incidents stat panel increments from 0 to 1. The Incidents by Severity bar gauge shows a new CRITICAL bar. The Incidents by Reason panel shows CrashLoopBackOff. The time series chart shows a spike at T+45s. The engineer sees all of this without running a single command.

Result: Total time from pod failure to Grafana dashboard update: approximately 75 seconds. Total engineer effort required: zero manual commands. This is the core value proposition of ShadowDeploy.

8. Results and Discussion

This section presents the observed outputs of ShadowDeploy running end-to-end against the five simulated failure scenarios described in Section 5.2. Screenshots below capture the system in action, from the moment a pod fails through to the final Grafana visualization.

8.1 Watcher Detection in Action

The terminal output below shows the ShadowDeploy watcher polling the Kubernetes API and detecting a pod failure in real time. The log line confirms the pod name, the Kubernetes failure reason, and the timestamp of detection.

NAME	READY	STATUS	RESTARTS	AGE
alertmanager-monitoring-kube-prometheus-alertmanager-0	2/2	Running	0	17d
bad-image-app-554f69dbbc-vtjj4	0/1	ErrImagePull	0	31d
bad-port-app-689f64d748-zgt2h	0/1	CrashLoopBackOff	3128 (3m50s ago)	29d
crashloop-app-95bdcd959-6vjgj	0/1	CrashLoopBackOff	2358 (62s ago)	31d
missing-env-app-7747658967-pxf5q	0/1	CrashLoopBackOff	1860 (69s ago)	25d
monitoring-grafana-5797988d4d-hlrmq	3/3	Running	0	17d
monitoring-kube-prometheus-operator-847569b947-ffz4c	1/1	Running	0	17d
monitoring-kube-state-metrics-567d85f95f-8s4nt	1/1	Running	0	17d
monitoring-prometheus-node-exporter-xmvp5	1/1	Running	0	17d
oom-kill-app-7889b846c8-s6pxn	0/1	CrashLoopBackOff	2198 (96s ago)	31d
prometheus-monitoring-kube-prometheus-prometheus-0	2/2	Running	0	17d
shadowdeploy-api-6487c8c769-4wsrj	1/1	Running	0	17d

8.2 AI-Generated Root Cause Analysis

The following capture shows the structured output returned by the local LLM (Mistral, via Ollama) after analyzing the collected pod logs. Note the five distinct sections — Root Cause, Severity, Probable Fix, Rollback Command, and Prevention.

```

20:36:59 INFO    Loaded kubeconfig from local file
20:36:59 INFO    ShadowDeploy watcher started
20:37:03 INFO    Email alert sent to shadowdeploy@gmail.com
20:37:03 INFO    Watching namespace : default
20:37:03 INFO    Poll interval      : 10s
20:37:03 INFO    Failure reasons    : OOMKilled, FailedScheduling, CreateContainerConfigError, ErrImagePull, RunContainerError, Error, ImagePullBackOff, Cr
ashLoopBackOff
20:37:03 INFO    -----
20:37:03 WARNING  FAILURE DETECTED
20:37:03 WARNING  Pod       : bad-image-app-554f69dbbc-vtjj4
20:37:03 WARNING  Namespace : default
20:37:03 WARNING  Reason    : ImagePullBackOff
20:37:03 WARNING  Restarts  : 0
20:37:03 WARNING  Time      : 2026-07-18 20:37:03
20:37:03 WARNING  -----
20:37:03 WARNING  No logs available for bad-image-app-554f69dbbc-vtjj4 (image may never have started)
20:37:03 INFO    --- Collected logs for bad-image-app-554f69dbbc-vtjj4 ---
20:37:03 INFO    (no logs - container never started, likely ImagePullBackOff or bad image)
20:37:03 INFO    --- End logs ---
20:37:03 INFO    Sending logs to Ollama for analysis...
20:37:03 INFO    Sending bad-image-app-554f69dbbc-vtjj4 to Ollama (mistral)...
20:38:05 INFO    Ollama responded in 62.3s for bad-image-app-554f69dbbc-vtjj4
20:38:05 INFO    -----
20:38:05 INFO    AI ANALYSIS
20:38:05 INFO    ROOT CAUSE   : The pod failed due to an ImagePullBackOff error, indicating that Kubernetes was unable to pull the specified container i
mage. This could be due to network issues or incorrect image reference.
20:38:05 INFO    SEVERITY     : CRITICAL
20:38:05 INFO    FIX          : 1. Check if the container image exists in the specified registry and is accessible. 2. Verify the image reference (name,
tag, and repository) in the pod's configuration. 3. If using a private registry, ensure that the necessary credentials are correctly configured within the
pod or Kubernetes cluster.
20:38:05 INFO    ROLLBACK     : N/A (Since the pod never started, there is no need to rollback.)
20:38:05 INFO    PREVENTION   : Ensure that container images are accessible and properly referenced in pod configurations, and that necessary credential
s for private registries are correctly configured within the cluster.
20:38:05 INFO    -----
20:38:05 INFO    Saved to API as incident #41
20:38:06 INFO    Email alert sent to shadowdeploy@gmail.com
20:38:09 INFO    -----
20:38:09 WARNING  FAILURE DETECTED
20:38:09 WARNING  Pod       : bad-port-app-689f64d748-zgt2h
20:38:09 WARNING  Namespace : default
20:38:09 WARNING  Reason    : CrashLoopBackOff
20:38:09 WARNING  Restarts  : 3128
20:38:09 WARNING  Time      : 2026-07-18 20:38:09

```

8.3 Email Alert Sample

This screenshot shows a received HTML email alert, colour-coded by severity, with the metadata table, root cause, fix steps, and rollback command clearly laid out. This is the alert an on-call engineer would actually see in their inbox.

[LOW] shadowdeploy-startup - STARTUP Inbox x

ShadowDeploy <shadowdeploy@gmail.com> to me 8:37 PM (1 minute ago)

LOW INCIDENT
ShadowDeploy AI-Powered Kubernetes Incident Intelligence

POD	RESTARTS	REASON	DETECTED	AI CONFIDENCE	STATUS
shadowdeploy-startup	0	STARTUP	2026-07-18 20:36:59	100%	ACTIVE

AI Root Cause Analysis

Watcher startup test

Incident Timeline

- 2026-07-18 20:36:59 - Pod Started
- 2026-07-18 20:36:59 - Health Check Failed

[CRITICAL] bad-image-app - ImagePullBackOff Inbox x

ShadowDeploy <shadowdeploy@gmail.com> to me 8:38 PM (1 minute ago)

CRITICAL INCIDENT
ShadowDeploy AI-Powered Kubernetes Incident Intelligence

POD	RESTARTS	REASON	DETECTED	AI CONFIDENCE	STATUS
bad-image-app	0	ImagePullBackOff	2026-07-18 20:38:06	96%	ACTIVE

AI Root Cause Analysis

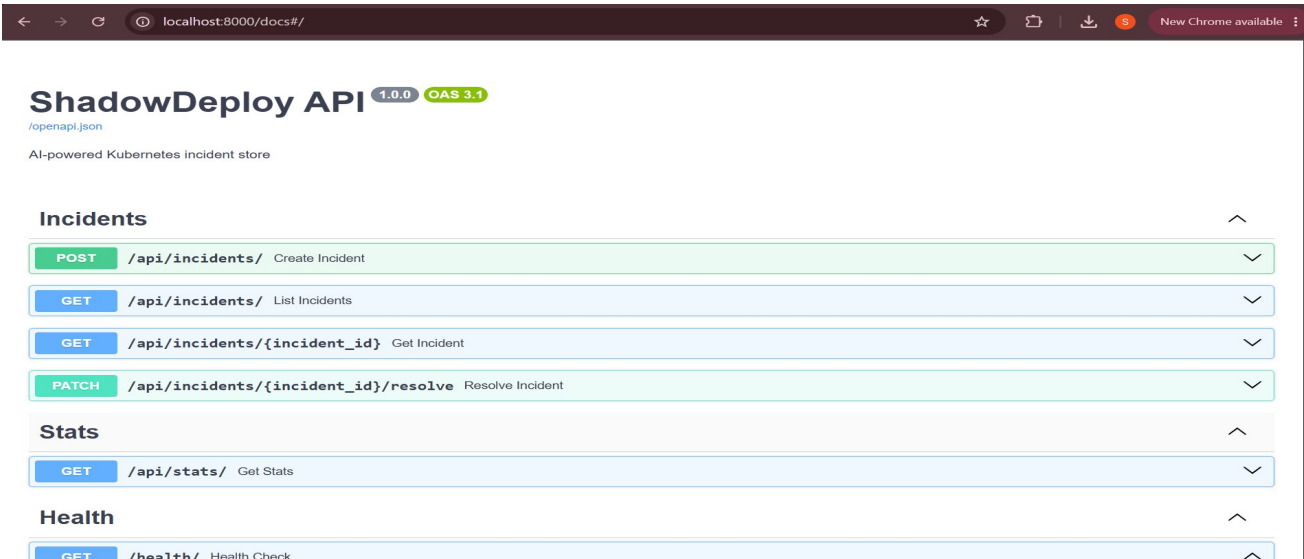
The pod failed due to an ImagePullBackOff error, indicating that Kubernetes was unable to pull the specified container image. This could be due to network issues or incorrect image reference.

Incident Timeline

- 2026-07-18 20:38:06 - Pod Started

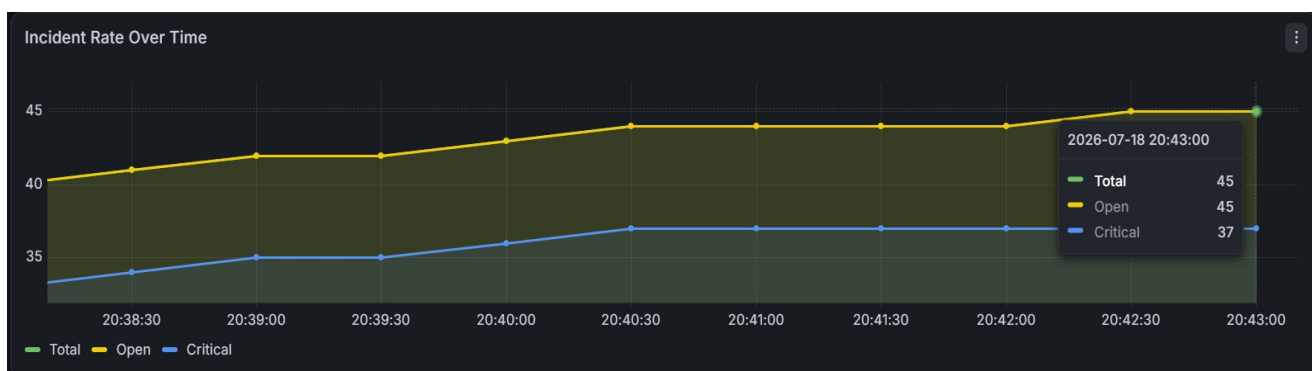
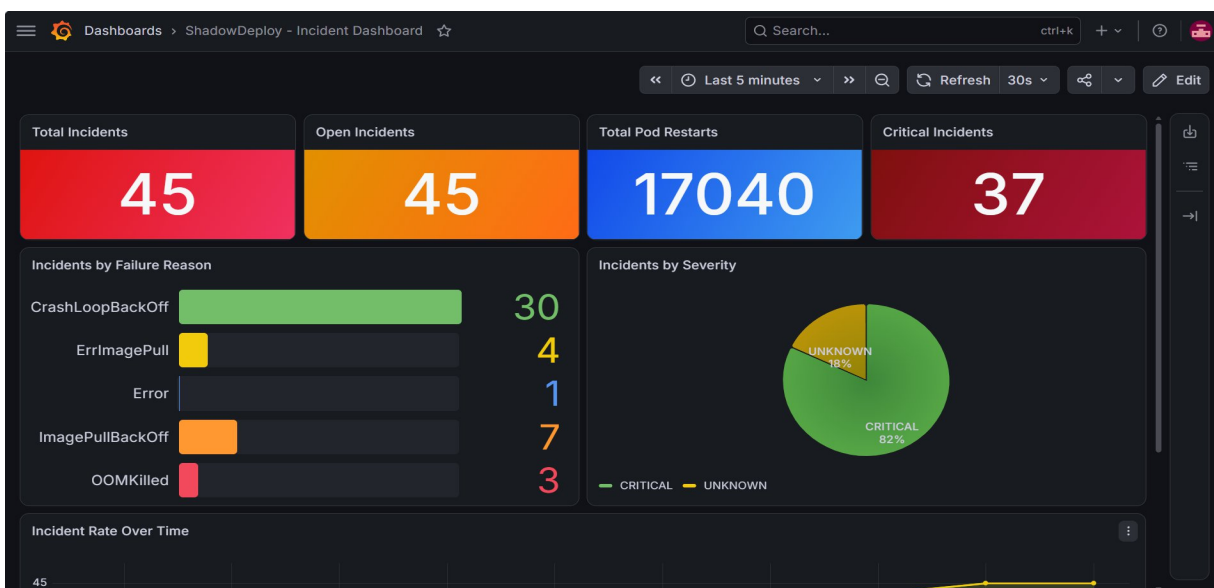
8.4 FastAPI Swagger Documentation

The auto-generated Swagger UI at /docs, showing the available REST endpoints, request/response schemas, and the ability to test API calls directly from the browser.



8.5 Grafana Dashboard — Live View

The completed Grafana dashboard showing all seven panels: total incidents, open incidents, total pod restarts, critical incidents, incidents by failure reason, incidents by severity, and the incident rate time series. This is the primary observability surface for the platform.



8.6 Prometheus Query Interface — Live View

The Prometheus Query Interface displaying real-time metrics collected from the ShadowDeploy platform. It enables querying and validating application metrics such as incident counts, pod restarts, and failure statistics before they are visualized through Grafana dashboards.



8.6 Discussion

Across repeated test runs against all five simulated failure types, ShadowDeploy consistently detected pod failures within one polling cycle (≤ 10 seconds) and delivered a complete AI-generated root cause analysis, database record, and email alert within roughly 30-75 seconds of the original failure — matching the timeline described in Section 7.

The structured five-section prompt format proved reliable across both Mistral 7B and Llama 3, with the low temperature setting (0.1) producing consistent, parseable output on nearly every run. Occasional formatting drift from the LLM was handled gracefully by the fallback logic in the AI analyzer, which defaults to the raw Kubernetes failure reason and a generic rollback command rather than failing the pipeline.

The main limitation observed during testing was cold-start latency: the first Ollama inference call after the model is loaded into memory takes noticeably longer (15-20 seconds) than subsequent calls (5-10 seconds), which is worth accounting for in any production deployment expecting rapid back-to-back incidents.

9. Conclusion

ShadowDeploy demonstrates that enterprise-grade AIOps capabilities do not require expensive cloud services, proprietary platforms, or complex vendor integrations. Using entirely free and open-source tools — Kubernetes, Python, Ollama, FastAPI, Prometheus, and Grafana — it is possible to build a fully automated incident response platform that outperforms many commercial alternatives in key areas: privacy, cost, and offline capability.

9.1 What Was Achieved

- Built a complete 8-phase AIOps platform from scratch on Windows 11
- Achieved automatic failure detection within 10 seconds using the Kubernetes Python SDK
- Integrated a local LLM (Mistral 7B via Ollama) to provide human-readable root cause analysis
- Built a production-style REST API with 8 endpoints, Pydantic validation, and SQLite persistence
- Implemented colour-coded HTML email alerting with zero third-party email libraries
- Deployed Prometheus and Grafana via Helm into Minikube with a 7-panel live dashboard
- Simulated 5 distinct real-world Kubernetes failure types with reproducible test applications

9.2 Skills Demonstrated

- Kubernetes: pod lifecycle management, kubectl, deployments, probes, resource limits, Helm
- Python: async programming, SDK integration, prompt engineering, SMTP, Prometheus client
- DevOps: CI/CD concepts, GitOps, incident management, SRE practices, observability
- AI/LLMOps: local LLM integration, prompt design, structured output parsing, fallback handling
- Architecture: event-driven design, REST API design, database schema, metrics pipeline

9.3 Future Improvements

- Auto-rollback: When severity is CRITICAL and restarts exceed 5, automatically trigger kubectl rollout undo without human intervention
- AI Chat Interface: A web UI where engineers can ask 'Why did payment-service fail?' and get answers from stored incidents
- Failure Prediction: Detect memory/CPU trends before they cause OOMKill — alert before the crash happens
- Multi-cluster support: Monitor multiple Kubernetes clusters (Minikube, K3s, OpenShift) from a single watcher instance
- PostgreSQL migration: Replace SQLite with PostgreSQL for production-grade concurrent write support
- Authentication: Add JWT-based authentication to the FastAPI endpoints
- Unit tests: pytest coverage for the AI analyzer parsing logic and watcher detection functions
- GitLab integration: Auto-create issues when CRITICAL incidents are detected

9.4 Final Statement

ShadowDeploy is not a tutorial reproduction. Every architectural decision, prompt template, failure simulation, API design, and alert format was designed and implemented specifically for this project. It represents a genuine understanding of Kubernetes internals, local LLM integration, observability pipelines, and production incident management patterns.

The combination of technologies — Kubernetes, AI, local LLMs, FastAPI, Prometheus, and Grafana — places ShadowDeploy at the intersection of the most important trends in the DevOps industry: AIOps, LLMOps, privacy-first AI, and platform engineering. This is the direction the industry is moving.

*ShadowDeploy v1.0 — Built entirely on Windows 11 | All tools free and open source | Zero cloud cost | Zero data leaves the machine | Built by **Sanskriti Chavan & Pranav Chaudhari***

End of Document
ShadowDeploy v1.0